

Analytical Problems.

Name

NAME: N. Jaffrin

Reg. No: 192521200.

1) Evaluate the following postfix expression using a stack. Show the stack contents after every operation.

$$23 * 54 * + 9 -$$

character	stack	output.
2	-	2
3	-	2, 3
*	*	2 3 *
5	*	2 3 * 5
4	-	2 3 * 5 4
*	*	2 3 * 5 4 *
+	+	2 3 * 5 4 * +
9		2 3 * 5 4 * + 9
-		2 3 * 5 4 * + 9 -

$$\begin{array}{|c|} \hline 2 \\ \hline 3 \\ \hline \end{array} \Rightarrow \begin{array}{|c|} \hline 14 \\ \hline 5 \\ \hline 6 \\ \hline \end{array} \begin{array}{|c|} \hline 20 \\ \hline 6 \\ \hline \end{array} \begin{array}{|c|} \hline 26 \\ \hline 9 \\ \hline \end{array} \begin{array}{|c|} \hline 17 \\ \hline \end{array}$$

Final answer = 17

3/8/20

- 2) Design an algorithm to evaluate any postfix expression using a stack.

Algorithm to Evaluate a postfix Expression

- 1) Create an empty stack.
- 2) Scan the postfix expression left to right.
- 3) If the token is an operand, push it onto the stack.
- 4) If the token is an operator :
 - * Pop the two elements
 - * Compute $\text{result} = \text{op1} < \text{operator} > \text{op2}$
 - * Push result back onto the stack.
- 5) Repeat steps 3-4 until all tokens are processed.
- 6) At the end, the stack should contain exactly one value - pop it. That is the final result.

Pseudocode :

for each token in postfix-expression :

if token is operand :

stack.push(token)

else :

op2 = stack.pop()

op1 = stack.pop()

result = apply(token, op1, op2)

stack.push(result)

return stack.pop()

Why stacks are more suitable than queues

- * Postfix evaluation always needs the most recently computed / most recently seen operand inherently a last-in-first-out (LIFO) requirement.

- * A stack's `pop()` naturally gives you the most recent elements first, matching this need directly.

- * A queue is FIFO - `dequeue()` would give you the oldest operands, which are usually not the correct operands for the current operator.

- * Postfix evaluation needs the most recently pushed operands for each operator.

